

COURSEWORK 2 | WORLD OF ZUUL

The Last Lantern

Programming Practice and Applications - implementation report

Course: 4CCS1PPA Programming Practice and Applications

Submission: Executable BlueJ-compatible JAR and report with source appendix

Game overview

The Last Lantern is a text adventure set in the abandoned Lantern Keep. The player explores nine linked locations, collects and trades objects, and must relight the moon beacon before the keep's magic disappears. The required route is deterministic: obtain bread, trade it to the dwarf for the star wand, reach the observatory, and use the wand on the beacon. A roaming clockwork raven and an optional random transporter make the world feel active without making the winning walkthrough depend on chance.

The implementation separates console parsing, commands, rooms, items, player state, and non-player characters. It uses typed collections, immutable value objects where appropriate, deterministic display ordering, and injected input, output, and randomness for repeatable testing.

Base tasks completed

1. **Several rooms.** Nine named rooms form Lantern Keep, with ordinary routes kept bidirectional and the optional Shimmer Chamber on a side branch.
2. **Player movement.** The player travels with `go <direction>`. Invalid exits leave all location and history state unchanged.
3. **Room items.** Each room owns an unrestricted item map. Portable examples include bread, rope, and lantern; fixed scenery includes the lectern, cauldron, statue, anvil, and beacon.
4. **Weighted inventory.** Every item has a positive weight. Player enforces a 6 kg total capacity, accepts an item exactly at the limit, and rejects an item that would exceed it without changing room state.

Base tasks completed (continued)

5. **Recognised win.** The game ends only when a player carrying the star wand enters the Moon Observatory and issues `use wand beacon`. It prints an explicit victory banner and terminates the play loop.
6. **Back command.** Player stores successful travel in a Deque, so repeated back commands walk through the complete history. Failed moves create no history. Transport arrival replaces the transient chamber, so back returns to the last meaningful room.
7. **New commands.** Eleven commands were added beyond `go/help/quit`: `back`, `look`, `take`, `drop`, `inventory`, `examine`, `talk`, `give`, `use`, `hint`, and `wait`. This exceeds the requirement even if `back` is counted separately.

Challenge tasks completed

- **Moving characters.** A clockwork raven occupies a room and moves through a random ordinary exit when time advances. The quest-critical dwarf is deliberately stationary so random movement cannot break the quest.
- **Three-word parser.** Command and Parser retain a third word and detect extra input. The quest uses `give bread dwarf` and `use wand beacon`.
- **Magic transporter.** Entering the Shimmer Chamber immediately selects a random ordinary room, excluding the chamber and the room just left. The destination and updated room description are announced.

Additional challenges

- A context-sensitive hint system gives the next objective without revealing unrelated state.
- Input is case-insensitive, blank lines and EOF are safe, and commands longer than three words receive a precise error rather than being silently truncated.
- A dependency-free regression suite and injectable collaborators make random and console behaviour reproducible.

Code quality considerations

Coupling

Parser knows only Command and CommandWords; it has no dependency on rooms, items, or quest rules. Game coordinates domain objects through small methods such as Room.getExit, Player.addItem, and GameCharacter.moveRandomly. This keeps changes to input syntax or room storage local rather than spreading them across the application.

Cohesion

Each class has one focused purpose. Room owns exits, room items, and occupants; Player owns location history and inventory capacity; Item is an immutable description of a weighted object; and GameCharacter owns character placement and movement. Methods operate on the data that belongs to their class.

Responsibility-driven design

Rules are assigned to the object with the necessary information. Player decides whether an item fits and computes carried weight. Room adds or removes its own contents. Game remains the controller that interprets player intent, performs quest transactions, advances the world, and recognises victory.

Maintainability

CommandWords is the single vocabulary source; LinkedHashMap gives stable output; constants name the carry limit; constructors validate invalid state; and no raw collections or unchecked casts remain. Reader, PrintStream, and Random injection support tests without special production branches. Java 8 compilation with all lint checks enabled completes with zero warnings.

Verification

- Nine included regression groups cover parsing, multiple/fixed items, carry limits, back history, moving characters, transport, item rules, argument validation, dynamic hints, and the complete win path.
- Sixteen independent adversarial groups cover EOF, invalid arity, failed movement, repeated take/drop/give, full inventory rewards, early or incorrect use, transporter history, and random-state fuzzing.
- The winning route passed for 500 random seeds; two fresh java -jar runs also reached the win banner and exited with status 0.

Winning walkthrough

Start a new game and enter the following commands in order. This route does not enter the random transporter and is independent of the raven's position.

1. `go north`
2. `go east`
3. `take bread`
4. `go west`
5. `go north`
6. `go east`
7. `give bread dwarf`
8. `go west`
9. `go north`
10. `use wand beacon`

Expected result: silver fire relights the beacon, the game prints 'YOU HAVE WON THE LAST LANTERN!', and the command loop ends.

Known bugs and deliberate limits

- No known functional bugs remain after the regression, adversarial, multi-seed, and executable-JAR tests described above.
- Item and character identifiers intentionally use one word so a three-word command is unambiguous. Descriptions may contain any number of words.
- The moving raven is atmospheric and cannot block progress. Random transport is an optional exploration feature and is not required to complete the game.

Submission contents

The-Last-Lantern.jar is an executable, BlueJ-compatible project JAR whose manifest names Game as Main-Class. The source appendix that follows contains every Java class, including the regression test class, and is excluded from the four-page report limit.

Source Appendix

Complete source of all nine Java classes. This appendix is excluded from the four-page report limit.

Command.java

```
/**
 * Represents one command entered by the player.
 *
 * A command can contain up to three words. The third word supports commands
 * such as "give bread dwarf". The parser also records whether extra words
 * were supplied so that the game can report the error instead of silently
 * ignoring input.
 */
public class Command
{
    private final String commandWord;
    private final String secondWord;
    private final String thirdWord;
    private final boolean extraWords;

    /**
     * Backwards-compatible constructor for one- and two-word commands.
     */
    public Command(String firstWord, String secondWord)
    {
        this(firstWord, secondWord, null, false);
    }

    /**
     * Creates a parsed command.
     */
    public Command(String firstWord, String secondWord, String thirdWord,
                  boolean extraWords)
    {
        this.commandWord = firstWord;
        this.secondWord = secondWord;
        this.thirdWord = thirdWord;
        this.extraWords = extraWords;
    }

    public String getCommandWord()
    {
        return commandWord;
    }

    public String getSecondWord()
    {
        return secondWord;
    }

    public String getThirdWord()
    {
        return thirdWord;
    }

    public boolean isUnknown()
    {

```

```

        return commandWord == null;
    }

    public boolean hasSecondWord()
    {
        return secondWord != null;
    }

    public boolean hasThirdWord()
    {
        return thirdWord != null;
    }

    public boolean hasExtraWords()
    {
        return extraWords;
    }
}

```

CommandWords.java

```

import java.io.PrintStream;
import java.util.Arrays;
import java.util.Collections;
import java.util.LinkedHashSet;
import java.util.Set;

/**
 * Stores the command vocabulary understood by the game.
 */
public class CommandWords
{
    private static final Set<String> VALID_COMMANDS =
        Collections.unmodifiableSet(new LinkedHashSet<String>(Arrays.asList(
            "go", "back", "look", "take", "drop", "inventory",
            "examine", "talk", "give", "use", "hint", "wait",
            "help", "quit"
        )));

    /**
     * Returns true when the supplied word is a recognised command.
     */
    public boolean isCommand(String word)
    {
        return word != null && VALID_COMMANDS.contains(word);
    }

    /**
     * Prints all commands to the supplied output stream.
     */
    public void showAll(PrintStream output)
    {
        boolean first = true;
        for (String command : VALID_COMMANDS) {
            if (!first) {
                output.print(" ");
            }
            output.print(command);
            first = false;
        }
    }
}

```

```

    }
    output.println();
}

/**
 * Retained for compatibility with the original BlueJ project.
 */
public void showAll()
{
    showAll(System.out);
}
}

```

Parser.java

```

import java.io.BufferedReader;
import java.io.IOException;
import java.io.InputStreamReader;
import java.io.PrintStream;
import java.io.Reader;
import java.util.Locale;

/**
 * Reads and validates console commands for the adventure game.
 */
public class Parser
{
    private final CommandWords commands;
    private final BufferedReader reader;
    private final PrintStream output;

    /**
     * Creates a parser connected to the console.
     */
    public Parser()
    {
        this(new InputStreamReader(System.in), System.out);
    }

    /**
     * Creates a parser using injectable input and output. This constructor
     * keeps the parser testable without changing normal console behaviour.
     */
    public Parser(Reader input, PrintStream output)
    {
        if (input == null || output == null) {
            throw new IllegalArgumentException("Input and output are required.");
        }
        this.commands = new CommandWords();
        this.reader = new BufferedReader(input);
        this.output = output;
    }

    /**
     * Reads one line and returns a command containing at most three words.
     * End-of-file is treated as a clean quit so scripted runs cannot crash.
     */
    public Command getCommand()
    {

```

```

output.print("> ");

final String inputLine;
try {
    inputLine = reader.readLine();
}
catch (IOException exception) {
    output.println("There was an error while reading: "
        + exception.getMessage());
    return new Command(null, null, null, false);
}

if (inputLine == null) {
    return new Command("quit", null, null, false);
}

String trimmed = inputLine.trim().toLowerCase(Locale.ENGLISH);
if (trimmed.length() == 0) {
    return new Command(null, null, null, false);
}

String[] words = trimmed.split("\\s+");
String firstWord = words[0];
String secondWord = words.length > 1 ? words[1] : null;
String thirdWord = words.length > 2 ? words[2] : null;
boolean extraWords = words.length > 3;

if (!commands.isCommand(firstWord)) {
    firstWord = null;
}
return new Command(firstWord, secondWord, thirdWord, extraWords);
}

public void showCommands()
{
    commands.showAll(output);
}
}

```

Item.java

```

import java.util.Locale;

/**
 * An object that can be placed in a room or carried by the player.
 */
public class Item
{
    private final String name;
    private final String description;
    private final int weight;
    private final boolean portable;

    public Item(String name, String description, int weight, boolean portable)
    {
        if (name == null || name.trim().length() == 0) {
            throw new IllegalArgumentException("An item must have a name.");
        }
        if (description == null || description.trim().length() == 0) {

```



```

        throw new IllegalArgumentException("An item needs a description.");
    }
    if (weight <= 0) {
        throw new IllegalArgumentException("Item weight must be positive.");
    }
    this.name = name.trim().toLowerCase(Locale.ENGLISH);
    this.description = description.trim();
    this.weight = weight;
    this.portable = portable;
}

public String getName()
{
    return name;
}

public String getDescription()
{
    return description;
}

public int getWeight()
{
    return weight;
}

public boolean isPortable()
{
    return portable;
}

public String getDisplayDescription()
{
    String status = portable ? "portable" : "fixed in place";
    return name + " - " + description + " (" + weight + " kg, "
        + status + ")";
}
}

```

Room.java

```

import java.util.ArrayList;
import java.util.Collection;
import java.util.Collections;
import java.util.LinkedHashMap;
import java.util.List;
import java.util.Locale;
import java.util.Map;

/**
 * One location in the game world.
 *
 * A room owns its exits, items, and characters. Linked maps keep display
 * order deterministic while allowing any number of entries.
 */
public class Room
{
    private final String name;
    private final String description;

```

```

private final boolean transporter;
private final Map<String, Room> exits;
private final Map<String, Item> items;
private final Map<String, GameCharacter> characters;

/**
 * Retains the single-description constructor used by the starter code.
 */
public Room(String description)
{
    this(description, description, false);
}

public Room(String name, String description)
{
    this(name, description, false);
}

public Room(String name, String description, boolean transporter)
{
    if (name == null || name.trim().length() == 0) {
        throw new IllegalArgumentException("A room must have a name.");
    }
    if (description == null || description.trim().length() == 0) {
        throw new IllegalArgumentException("A room needs a description.");
    }
    this.name = name.trim();
    this.description = description.trim();
    this.transporter = transporter;
    this.exits = new LinkedHashMap<String, Room>();
    this.items = new LinkedHashMap<String, Item>();
    this.characters = new LinkedHashMap<String, GameCharacter>();
}

public String getName()
{
    return name;
}

public void setExit(String direction, Room neighbour)
{
    if (direction == null || direction.trim().length() == 0
        || neighbour == null) {
        throw new IllegalArgumentException("An exit needs a direction and room.");
    }
    exits.put(normalise(direction), neighbour);
}

public Room getExit(String direction)
{
    return direction == null ? null : exits.get(normalise(direction));
}

/**
 * Returns a snapshot so callers cannot alter this room's exits.
 */
public List<Room> getNeighbouringRooms()
{
    return Collections.unmodifiableList(
        new ArrayList<Room>(exits.values()));
}

```

```

}

public String getShortDescription()
{
    return description;
}

public String getLongDescription()
{
    StringBuilder text = new StringBuilder();
    text.append("You are ").append(description).append(".\n");
    text.append(getExitString()).append("\n");
    text.append(getItemString()).append("\n");
    text.append(getCharacterString());
    return text.toString();
}

public void addItem(Item item)
{
    if (item == null) {
        throw new IllegalArgumentException("Cannot add a null item.");
    }
    items.put(item.getName(), item);
}

public Item getItem(String itemName)
{
    return itemName == null ? null : items.get(normalise(itemName));
}

public Item removeItem(String itemName)
{
    return itemName == null ? null : items.remove(normalise(itemName));
}

public Collection<Item> getItems()
{
    return Collections.unmodifiableCollection(items.values());
}

public GameCharacter getCharacter(String characterName)
{
    return characterName == null
        ? null : characters.get(normalise(characterName));
}

public Collection<GameCharacter> getCharacters()
{
    return Collections.unmodifiableCollection(characters.values());
}

void addCharacter(GameCharacter character)
{
    characters.put(character.getName(), character);
}

void removeCharacter(GameCharacter character)
{
    if (character != null) {
        characters.remove(character.getName());
    }
}

```

```

    }
}

public boolean isTransporter()
{
    return transporter;
}

private String getExitString()
{
    StringBuilder text = new StringBuilder("Exits:");
    if (exits.isEmpty()) {
        return text.append(" none").toString();
    }
    for (String direction : exits.keySet()) {
        text.append(" ").append(direction);
    }
    return text.toString();
}

private String getItemString()
{
    if (items.isEmpty()) {
        return "Items: none";
    }
    StringBuilder text = new StringBuilder("Items:");
    for (Item item : items.values()) {
        text.append(" ").append(item.getName())
            .append("[").append(item.getWeight()).append("kg");
        if (!item.isPortable()) {
            text.append(", fixed");
        }
        text.append("]");
    }
    return text.toString();
}

private String getCharacterString()
{
    if (characters.isEmpty()) {
        return "Characters: none";
    }
    StringBuilder text = new StringBuilder("Characters:");
    for (GameCharacter character : characters.values()) {
        text.append(" ").append(character.getName());
    }
    return text.toString();
}

private static String normalise(String text)
{
    return text.trim().toLowerCase(Locale.ENGLISH);
}
}

```

Player.java

```

import java.util.ArrayDeque;
import java.util.Collection;

```

```

import java.util.Collections;
import java.util.Deque;
import java.util.LinkedHashMap;
import java.util.Locale;
import java.util.Map;

/**
 * Holds player state: location, travel history, and carried items.
 */
public class Player
{
    private Room currentRoom;
    private final Deque<Room> roomHistory;
    private final Map<String, Item> inventory;
    private final int maximumCarryWeight;

    public Player(Room startingRoom, int maximumCarryWeight)
    {
        if (startingRoom == null) {
            throw new IllegalArgumentException("A player needs a starting room.");
        }
        if (maximumCarryWeight <= 0) {
            throw new IllegalArgumentException("Carry weight must be positive.");
        }
        this.currentRoom = startingRoom;
        this.maximumCarryWeight = maximumCarryWeight;
        this.roomHistory = new ArrayDeque<Room>();
        this.inventory = new LinkedHashMap<String, Item>();
    }

    public Room getCurrentRoom()
    {
        return currentRoom;
    }

    /**
     * Moves to a room and records the previous room for the back command.
     */
    public void moveTo(Room destination)
    {
        if (destination == null) {
            throw new IllegalArgumentException("Destination cannot be null.");
        }
        roomHistory.push(currentRoom);
        currentRoom = destination;
    }

    /**
     * Replaces a transient room with its teleport destination. No additional
     * history entry is created, so back returns to the last meaningful room.
     */
    public void teleportTo(Room destination)
    {
        if (destination == null) {
            throw new IllegalArgumentException("Destination cannot be null.");
        }
        currentRoom = destination;
    }
}

```

```

* Moves one step back through the full travel history.
*/
public boolean goBack()
{
    if (roomHistory.isEmpty()) {
        return false;
    }
    currentRoom = roomHistory.pop();
    return true;
}

public boolean canCarry(Item item)
{
    return item != null && item.isPortable()
        && getCurrentWeight() + item.getWeight() <= maximumCarryWeight;
}

public boolean addItem(Item item)
{
    if (!canCarry(item) || inventory.containsKey(item.getName())) {
        return false;
    }
    inventory.put(item.getName(), item);
    return true;
}

public Item removeItem(String itemName)
{
    return itemName == null ? null
        : inventory.remove(normalise(itemName));
}

public Item getItem(String itemName)
{
    return itemName == null ? null : inventory.get(normalise(itemName));
}

public boolean hasItem(String itemName)
{
    return getItem(itemName) != null;
}

public Collection<Item> getInventory()
{
    return Collections.unmodifiableCollection(inventory.values());
}

public int getCurrentWeight()
{
    int total = 0;
    for (Item item : inventory.values()) {
        total += item.getWeight();
    }
    return total;
}

public int getMaximumCarryWeight()
{
    return maximumCarryWeight;
}

```

```

private static String normalise(String text)
{
    return text.trim().toLowerCase(Locale.ENGLISH);
}
}

```

GameCharacter.java

```

import java.util.ArrayList;
import java.util.List;
import java.util.Locale;
import java.util.Random;

/**
 * A non-player character that occupies a room and may move independently.
 */
public class GameCharacter
{
    private final String name;
    private final String description;
    private final boolean mobile;
    private Room currentRoom;

    public GameCharacter(String name, String description, boolean mobile)
    {
        if (name == null || name.trim().length() == 0) {
            throw new IllegalArgumentException("A character must have a name.");
        }
        if (description == null || description.trim().length() == 0) {
            throw new IllegalArgumentException("A character needs a description.");
        }
        this.name = name.trim().toLowerCase(Locale.ENGLISH);
        this.description = description.trim();
        this.mobile = mobile;
    }

    public String getName()
    {
        return name;
    }

    public String getDescription()
    {
        return description;
    }

    public Room getCurrentRoom()
    {
        return currentRoom;
    }

    public boolean isMobile()
    {
        return mobile;
    }

    /**
     * Places the character in exactly one room.

```

```

*/
public void placeIn(Room room)
{
    if (room == null) {
        throw new IllegalArgumentException("Character room cannot be null.");
    }
    if (currentRoom != null) {
        currentRoom.removeCharacter(this);
    }
    currentRoom = room;
    currentRoom.addCharacter(this);
}

/**
 * Moves a mobile character through a random ordinary exit.
 * Transporter rooms are excluded because their magic targets the player.
 */
public boolean moveRandomly(Random random)
{
    if (!mobile || currentRoom == null || random == null) {
        return false;
    }

    List<Room> possibleRooms = new ArrayList<Room>();
    for (Room room : currentRoom.getNeighbouringRooms()) {
        if (!room.isTransporter()) {
            possibleRooms.add(room);
        }
    }
    if (possibleRooms.isEmpty()) {
        return false;
    }

    placeIn(possibleRooms.get(random.nextInt(possibleRooms.size())));
    return true;
}
}

```

Game.java

```

import java.io.InputStreamReader;
import java.io.PrintStream;
import java.io.Reader;
import java.util.ArrayList;
import java.util.List;
import java.util.Random;

/**
 * The Last Lantern - a text adventure based on World of Zuul.
 *
 * The player must feed a dwarf, receive a star wand, and relight the moon
 * beacon. This class builds the world and coordinates commands; item,
 * inventory, room, parser, and character responsibilities live in their own
 * focused classes.
 */
public class Game
{
    private static final int MAXIMUM_CARRY_WEIGHT = 6;

```



```

private final Parser parser;
private final PrintStream output;
private final Random random;
private final List<Room> rooms;
private final List<GameCharacter> characters;

private Player player;
private boolean dwarfHelped;
private boolean won;

/**
 * Creates a normal console game with non-deterministic challenge events.
 */
public Game()
{
    this(new InputStreamReader(System.in), System.out, new Random());
}

/**
 * Injectable constructor used by automated tests and scripted demos.
 */
public Game(Reader input, PrintStream output, Random random)
{
    this(new Parser(input, output), output, random);
}

/**
 * Creates a game from collaborating objects.
 */
public Game(Parser parser, PrintStream output, Random random)
{
    if (parser == null || output == null || random == null) {
        throw new IllegalArgumentException(
            "Parser, output, and random generator are required.");
    }
    this.parser = parser;
    this.output = output;
    this.random = random;
    this.rooms = new ArrayList<Room>();
    this.characters = new ArrayList<GameCharacter>();
    createWorld();
}

/**
 * Command-line entry point used by the executable JAR.
 */
public static void main(String[] args)
{
    new Game().play();
}

/**
 * Runs the game until the player wins or quits.
 */
public void play()
{
    printWelcome();

    boolean finished = false;
    while (!finished) {

```

```

        finished = processCommand(parser.getCommand());
    }

    if (won) {
        output.println("The Last Lantern burns again. Thank you for playing!");
    }
    else {
        output.println("Thank you for playing. Good bye.");
    }
}

/**
 * Exposes read-only state for tests and BlueJ inspection.
 */
public Room getCurrentRoom()
{
    return player.getCurrentRoom();
}

public boolean hasWon()
{
    return won;
}

/**
 * Creates nine linked rooms, their items, and two characters.
 */
private void createWorld()
{
    Room gatehouse = new Room(
        "Ruined Gatehouse",
        "at the ruined gatehouse beneath a moonless sky");
    Room courtyard = new Room(
        "Echo Courtyard",
        "in the echoing courtyard of Lantern Keep");
    Room library = new Room(
        "Dust Library",
        "inside a library where silver dust covers every book");
    Room kitchen = new Room(
        "Old Kitchen",
        "in an old kitchen that still smells of warm bread");
    Room greatHall = new Room(
        "Great Hall",
        "in the great hall beneath a cracked glass moon");
    Room armory = new Room(
        "Silent Armory",
        "inside the silent armory of the vanished lantern guard");
    Room dwarfDen = new Room(
        "Dwarf Workshop",
        "in a cramped workshop filled with brass shavings");
    Room observatory = new Room(
        "Moon Observatory",
        "at the moon observatory beside the dark beacon");
    Room transporter = new Room(
        "Shimmer Chamber",
        "inside a chamber whose floor shimmers like water", true);

    gatehouse.setExit("north", courtyard);
    gatehouse.setExit("east", library);

```

```

courtyard.setExit("south", gatehouse);
courtyard.setExit("west", library);
courtyard.setExit("east", kitchen);
courtyard.setExit("north", greatHall);
courtyard.setExit("down", transporter);

library.setExit("west", gatehouse);
library.setExit("east", courtyard);

kitchen.setExit("west", courtyard);

greatHall.setExit("south", courtyard);
greatHall.setExit("west", armory);
greatHall.setExit("east", dwarfDen);
greatHall.setExit("north", observatory);

armory.setExit("east", greatHall);
dwarfDen.setExit("west", greatHall);
observatory.setExit("south", greatHall);
transporter.setExit("up", courtyard);

gatehouse.addItem(new Item(
    "map", "a faded map showing the keep's main rooms", 1, true));
courtyard.addItem(new Item(
    "coin", "a moon-stamped copper coin", 1, true));
library.addItem(new Item(
    "lantern", "a small lantern with a cold blue flame", 2, true));
library.addItem(new Item(
    "lectern", "a stone lectern bolted to the floor", 45, false));
kitchen.addItem(new Item(
    "bread", "a fresh round loaf, surprisingly still warm", 1, true));
kitchen.addItem(new Item(
    "cauldron", "an iron cauldron far too large to move", 70, false));
greatHall.addItem(new Item(
    "statue", "a granite statue of the first lantern keeper", 200, false));
armory.addItem(new Item(
    "rope", "a coil of strong silk rope", 3, true));
armory.addItem(new Item(
    "shield", "a heavy shield engraved with a crescent", 5, true));
armory.addItem(new Item(
    "anvil", "a massive blacksmith's anvil", 300, false));
observatory.addItem(new Item(
    "beacon", "the fixed crystal beacon that must be relit", 500, false));

GameCharacter dwarf = new GameCharacter(
    "dwarf", "a hungry engineer guarding a star wand", false);
dwarf.placeIn(dwarfDen);
characters.add(dwarf);

GameCharacter raven = new GameCharacter(
    "raven", "a clockwork raven that patrols the keep", true);
raven.placeIn(library);
characters.add(raven);

rooms.add(gatehouse);
rooms.add(courtyard);
rooms.add(library);
rooms.add(kitchen);
rooms.add(greatHall);
rooms.add(armory);

```

```

rooms.add(dwarfDen);
rooms.add(observatory);
rooms.add(transporter);

player = new Player(gatehouse, MAXIMUM_CARRY_WEIGHT);
}

private void printWelcome()
{
    output.println();
    output.println("THE LAST LANTERN");
    output.println("Lantern Keep has gone dark. Find a way to relight the");
    output.println("moon beacon before the last of its magic disappears.");
    output.println("Type 'help' for commands and 'hint' for your next goal.");
    output.println();
    output.println(player.getCurrentRoom().getLongDescription());
}

/**
 * Dispatches one parsed command. True means the play loop should end.
 */
private boolean processCommand(Command command)
{
    if (command.hasExtraWords()) {
        output.println("Please use no more than three words.");
        return false;
    }
    if (command.isUnknown()) {
        output.println("I do not understand that command. Type 'help'.");
        return false;
    }

    String word = command.getCommandWord();
    if (word.equals("help")) {
        printHelp(command);
    }
    else if (word.equals("go")) {
        goRoom(command);
    }
    else if (word.equals("back")) {
        goBack(command);
    }
    else if (word.equals("look")) {
        look(command);
    }
    else if (word.equals("take")) {
        take(command);
    }
    else if (word.equals("drop")) {
        drop(command);
    }
    else if (word.equals("inventory")) {
        showInventory(command);
    }
    else if (word.equals("examine")) {
        examine(command);
    }
    else if (word.equals("talk")) {
        talk(command);
    }
}

```

```

    else if (word.equals("give")) {
        give(command);
    }
    else if (word.equals("use")) {
        return use(command);
    }
    else if (word.equals("hint")) {
        printHint(command);
    }
    else if (word.equals("wait")) {
        waitTurn(command);
    }
    else if (word.equals("quit")) {
        return quit(command);
    }
    return false;
}

private void printHelp(Command command)
{
    if (hasArguments(command)) {
        output.println("The help command takes no object.");
        return;
    }
    output.println("Goal: find the star wand and use it on the moon beacon.");
    output.println("Commands (items and characters use one-word names):");
    parser.showCommands();
    output.println("Examples: go north | take bread | give bread dwarf");
    output.println("        use wand beacon | examine map | back");
    output.println("Carry limit: " + player.getMaximumCarryWeight() + " kg.");
}

private void goRoom(Command command)
{
    if (!command.hasSecondWord()) {
        output.println("Go where?");
        return;
    }
    if (command.hasThirdWord()) {
        output.println("A direction must be one word.");
        return;
    }

    Room nextRoom = player.getCurrentRoom().getExit(command.getSecondWord());
    if (nextRoom == null) {
        output.println("There is no exit in that direction.");
        return;
    }

    Room previousRoom = player.getCurrentRoom();
    player.moveTo(nextRoom);
    enterCurrentRoom(previousRoom);
    moveCharacters();
    output.println(player.getCurrentRoom().getLongDescription());
}

private void goBack(Command command)
{
    if (hasArguments(command)) {
        output.println("The back command takes no object.");
    }
}

```

```

        return;
    }
    Room previousRoom = player.getCurrentRoom();
    if (!player.goBack()) {
        output.println("There is no previous room to return to.");
        return;
    }
    enterCurrentRoom(previousRoom);
    moveCharacters();
    output.println(player.getCurrentRoom().getLongDescription());
}

/**
 * Applies the transporter challenge whenever the player enters it.
 */
private void enterCurrentRoom(Room previousRoom)
{
    if (!player.getCurrentRoom().isTransporter()) {
        return;
    }

    List<Room> destinations = new ArrayList<Room>();
    for (Room room : rooms) {
        if (!room.isTransporter() && room != previousRoom) {
            destinations.add(room);
        }
    }
    Room destination = destinations.get(random.nextInt(destinations.size()));
    output.println("The chamber flashes. Space folds around you...");
    player.teleportTo(destination);
    output.println("You have been transported to " + destination.getName() + "!");
}

private void moveCharacters()
{
    for (GameCharacter character : characters) {
        character.moveRandomly(random);
    }
}

private void look(Command command)
{
    if (hasArguments(command)) {
        output.println("Use 'examine <item>' to inspect an object.");
        return;
    }
    output.println(player.getCurrentRoom().getLongDescription());
}

private void take(Command command)
{
    if (!requireOneObject(command, "Take what?")) {
        return;
    }

    Item item = player.getCurrentRoom().getItem(command.getSecondWord());
    if (item == null) {
        output.println("There is no " + command.getSecondWord() + " here.");
        return;
    }
}

```

```

    if (!item.isPortable()) {
        output.println("The " + item.getName() + " is fixed in place.");
        return;
    }
    if (!player.addItem(item)) {
        output.println("That would exceed your "
            + player.getMaximumCarryWeight() + " kg carry limit.");
        return;
    }

    player.getCurrentRoom().removeItem(item.getName());
    output.println("You take the " + item.getName() + ".");
    output.println("Carried weight: " + player.getCurrentWeight() + "/"
        + player.getMaximumCarryWeight() + " kg.");
}

private void drop(Command command)
{
    if (!requireOneObject(command, "Drop what?")) {
        return;
    }

    Item item = player.removeItem(command.getSecondWord());
    if (item == null) {
        output.println("You are not carrying " + command.getSecondWord() + ".");
        return;
    }
    player.getCurrentRoom().addItem(item);
    output.println("You drop the " + item.getName() + ".");
}

private void showInventory(Command command)
{
    if (hasArguments(command)) {
        output.println("The inventory command takes no object.");
        return;
    }
    if (player.getInventory().isEmpty()) {
        output.println("You are carrying nothing (0/"
            + player.getMaximumCarryWeight() + " kg).");
        return;
    }

    output.println("You are carrying:");
    for (Item item : player.getInventory()) {
        output.println("  " + item.getName() + " - " + item.getWeight() + " kg");
    }
    output.println("Total: " + player.getCurrentWeight() + "/"
        + player.getMaximumCarryWeight() + " kg.");
}

private void examine(Command command)
{
    if (!requireOneObject(command, "Examine what?")) {
        return;
    }

    String name = command.getSecondWord();
    Item item = player.getItem(name);
    if (item == null) {

```

```

        item = player.getCurrentRoom().getItem(name);
    }
    if (item != null) {
        output.println(item.getDisplayDescription());
        return;
    }

    GameCharacter character = player.getCurrentRoom().getCharacter(name);
    if (character != null) {
        output.println(character.getDescription());
        return;
    }
    output.println("There is no " + name + " here to examine.");
}

private void talk(Command command)
{
    if (!requireOneObject(command, "Talk to whom?")) {
        return;
    }

    GameCharacter character =
        player.getCurrentRoom().getCharacter(command.getSecondWord());
    if (character == null) {
        output.println("That character is not here.");
    }
    else if (character.getName().equals("dwarf") && !dwarfHelped) {
        output.println("The dwarf says: 'Bring me bread and I will trade'");
        output.println("you the star wand. Use: give bread dwarf.'");
    }
    else if (character.getName().equals("dwarf")) {
        output.println("The dwarf says: 'Take the wand north to the beacon.'");
    }
    else {
        output.println("The raven clicks: 'The hungry one guards starlight.'");
    }
}

/**
 * Three-word challenge command, for example "give bread dwarf".
 */
private void give(Command command)
{
    if (!command.hasSecondWord() || !command.hasThirdWord()) {
        output.println("Use: give <item> <character>.");
        return;
    }

    Item item = player.getItem(command.getSecondWord());
    if (item == null) {
        output.println("You are not carrying " + command.getSecondWord() + ".");
        return;
    }
    GameCharacter receiver =
        player.getCurrentRoom().getCharacter(command.getThirdWord());
    if (receiver == null) {
        output.println(command.getThirdWord() + " is not here.");
        return;
    }
    if (!receiver.getName().equals("dwarf") || !item.getName().equals("bread")) {

```



```

        output.println(receiver.getName() + " does not want the "
            + item.getName() + ".");
        return;
    }
    if (dwarfHelped) {
        output.println("The dwarf has already made the trade.");
        return;
    }

    player.removeItem(item.getName());
    dwarfHelped = true;
    Item wand = new Item(
        "wand", "a silver star wand humming with moonlight", 2, true);
    output.println("The dwarf devours the bread and gives you the star wand.");
    if (player.addItem(wand)) {
        output.println("The wand is now in your inventory.");
    }
    else {
        player.getCurrentRoom().addItem(wand);
        output.println("Your pack is too heavy, so the dwarf places the wand here.");
    }
}

private boolean use(Command command)
{
    if (!command.hasSecondWord()) {
        output.println("Use what?");
        return false;
    }

    Item item = player.getItem(command.getSecondWord());
    if (item == null) {
        output.println("You are not carrying " + command.getSecondWord() + ".");
        return false;
    }

    if (item.getName().equals("wand")) {
        if (!command.hasThirdWord()) {
            output.println("Use the wand on what?");
            return false;
        }
        if (command.getThirdWord().equals("beacon")
            && player.getCurrentRoom().getName().equals("Moon Observatory")) {
            output.println("Silver fire leaps from the wand into the beacon.");
            output.println("Light pours across the keep and the sky answers.");
            output.println("*** YOU HAVE WON THE LAST LANTERN! ***");
            won = true;
            return true;
        }
        output.println("The wand sparks, but nothing answers here.");
        return false;
    }

    if (item.getName().equals("lantern")) {
        if (command.hasThirdWord()) {
            output.println("The blue lantern has no effect on the "
                + command.getThirdWord() + ".");
        }
        else {
            output.println("The blue lantern reveals no hidden doorway.");
        }
    }
}

```

```

    }
}
else {
    String target = command.hasThirdWord()
        ? " on the " + command.getThirdWord() : "";
    output.println("Using the " + item.getName() + target
        + " has no useful effect.");
}
return false;
}

private void printHint(Command command)
{
    if (hasArguments(command)) {
        output.println("The hint command takes no object.");
        return;
    }
    if (!dwarfHelped && !player.hasItem("bread")) {
        Room breadRoom = findRoomContainingItem("bread");
        if (breadRoom != null) {
            output.println("Hint: the bread is in " + breadRoom.getName() + ".");
        }
        else {
            output.println("Hint: the dwarf still needs bread.");
        }
    }
    else if (!dwarfHelped) {
        output.println("Hint: the dwarf waits east of the great hall.");
    }
    else if (!player.hasItem("wand")) {
        Room wandRoom = findRoomContainingItem("wand");
        if (wandRoom != null) {
            output.println("Hint: the wand is in " + wandRoom.getName() + ".");
        }
        else {
            output.println("Hint: recover the star wand before going north.");
        }
    }
    else {
        output.println("Hint: go north from the great hall and use wand beacon.");
    }
}

private void waitTurn(Command command)
{
    if (hasArguments(command)) {
        output.println("The wait command takes no object.");
        return;
    }
    moveCharacters();
    output.println("You wait. Somewhere nearby, metal wings scrape stone.");
    output.println(player.getCurrentRoom().getLongDescription());
}

private boolean quit(Command command)
{
    if (hasArguments(command)) {
        output.println("Quit what?");
        return false;
    }
}

```

```

        return true;
    }

    private boolean requireOneObject(Command command, String missingMessage)
    {
        if (!command.hasSecondWord()) {
            output.println(missingMessage);
            return false;
        }
        if (command.hasThirdWord()) {
            output.println("That command accepts one object name.");
            return false;
        }
        return true;
    }

    private boolean hasArguments(Command command)
    {
        return command.hasSecondWord() || command.hasThirdWord();
    }

    private Room findRoomContainingItem(String itemName)
    {
        for (Room room : rooms) {
            if (room.getItem(itemName) != null) {
                return room;
            }
        }
        return null;
    }
}

```

GameTests.java

```

import java.io.ByteArrayOutputStream;
import java.io.PrintStream;
import java.io.StringReader;
import java.util.Random;

/**
 * Dependency-free regression tests. Run with: java -ea GameTests
 */
public class GameTests
{
    private static int testsRun;

    public static void main(String[] args) throws Exception
    {
        testThreeWordParserAndExtraWords();
        testRoomCanHoldMultipleItems();
        testCarryLimitAndFixedItems();
        testMultiStepBackHistory();
        testMovingCharacterChangesRoom();
        testTransporterMovesPlayer();
        testEndToEndItemRules();
        testArgumentValidationAndDroppedItemHints();
        testWinningWalkthrough();
        System.out.println("All " + testsRun + " tests passed.");
    }
}

```

```

private static void testThreeWordParserAndExtraWords() throws Exception
{
    ByteArrayOutputStream bytes = new ByteArrayOutputStream();
    PrintStream output = new PrintStream(bytes, true, "UTF-8");
    Parser parser = new Parser(
        new StringReader(" GIVE bread dwarf \nuse wand beacon now\n"),
        output);

    Command give = parser.getCommand();
    check("give".equals(give.getCommandWord()), "first word is normalised");
    check("bread".equals(give.getSecondWord()), "second word is parsed");
    check("dwarf".equals(give.getThirdWord()), "third word is parsed");
    check(!give.hasExtraWords(), "valid three-word command is accepted");

    Command tooLong = parser.getCommand();
    check(tooLong.hasExtraWords(), "fourth word is reported");
    pass();
}

private static void testRoomCanHoldMultipleItems()
{
    Room room = new Room("Test Room", "in a test room");
    room.addItem(new Item("apple", "a red apple", 1, true));
    room.addItem(new Item("table", "a fixed table", 20, false));

    check(room.getItems().size() == 2, "room holds multiple items");
    check(room.getItem("APPLE") != null, "item lookup ignores case");
    check(!room.getItem("table").isPortable(), "fixed item is represented");
    pass();
}

private static void testCarryLimitAndFixedItems()
{
    Room room = new Room("Start", "at the start");
    Player player = new Player(room, 4);
    Item exact = new Item("crate", "a four kilogram crate", 4, true);
    Item extra = new Item("coin", "a one kilogram coin", 1, true);
    Item fixed = new Item("door", "a fixed iron door", 100, false);

    check(player.addItem(exact), "exact carry limit is permitted");
    check(player.getCurrentWeight() == 4, "carried weight is summed");
    check(!player.addItem(extra), "weight above limit is rejected");
    check(!player.addItem(fixed), "non-portable item is rejected");
    check(player.removeItem("crate") == exact, "drop removes carried item");
    check(player.getCurrentWeight() == 0, "drop releases carry weight");
    pass();
}

private static void testMultiStepBackHistory()
{
    Room first = new Room("First", "in the first room");
    Room second = new Room("Second", "in the second room");
    Room third = new Room("Third", "in the third room");
    Player player = new Player(first, 5);

    player.moveTo(second);
    player.moveTo(third);
    check(player.goBack(), "first back succeeds");
    check(player.getCurrentRoom() == second, "first back reaches second room");
}

```

```

    check(player.goBack(), "second back succeeds");
    check(player.getCurrentRoom() == first, "second back reaches start");
    check(!player.goBack(), "back with empty history is safe");
    pass();
}

private static void testMovingCharacterChangesRoom()
{
    Room first = new Room("First", "in the first room");
    Room second = new Room("Second", "in the second room");
    first.setExit("east", second);
    GameCharacter raven = new GameCharacter("raven", "a test raven", true);
    raven.placeIn(first);

    check(raven.moveRandomly(new FixedRandom(0)), "mobile character moves");
    check(raven.getCurrentRoom() == second, "character reaches neighbouring room");
    check(first.getCharacter("raven") == null, "old room releases character");
    check(second.getCharacter("raven") == raven, "new room owns character");
    pass();
}

private static void testTransporterMovesPlayer() throws Exception
{
    GameRun run = runGame("go north\ngo down\nquit\n", new FixedRandom(0));
    check(run.text.contains("You have been transported to"),
        "transporter announces destination");
    check(!run.game.getCurrentRoom().isTransporter(),
        "player never remains in transporter room");
    pass();
}

private static void testEndToEndItemRules() throws Exception
{
    String script =
        "go east\n" +
        "take lectern\n" +
        "take lantern\n" +
        "go east\n" +
        "go north\n" +
        "go west\n" +
        "take shield\n" +
        "take rope\n" +
        "inventory\n" +
        "quit\n";
    GameRun run = runGame(script, new FixedRandom(0));

    check(run.text.contains("lectern is fixed in place"),
        "fixed room item cannot be taken");
    check(run.text.contains("exceed your 6 kg carry limit"),
        "overweight item is rejected");
    check(run.text.contains("Total: 5/6 kg"),
        "inventory reports exact carried weight");
    pass();
}

private static void testWinningWalkthrough() throws Exception
{
    String script =
        "go north\n" +
        "go east\n" +

```

```

        "take bread\n" +
        "go west\n" +
        "go north\n" +
        "go east\n" +
        "give bread dwarf\n" +
        "go west\n" +
        "go north\n" +
        "use wand beacon\n";
GameRun run = runGame(script, new Random(7));

check(run.game.hasWon(), "walkthrough sets win state");
check(run.text.contains("YOU HAVE WON THE LAST LANTERN"),
    "win is clearly announced");
check(!run.text.contains("I do not understand"),
    "walkthrough uses only valid commands");
pass();
}

private static void testArgumentValidationAndDroppedItemHints()
    throws Exception
{
    String script =
        "help object\n" +
        "go north\n" +
        "go east\n" +
        "take bread\n" +
        "go west\n" +
        "drop bread\n" +
        "hint\n" +
        "quit\n";
    GameRun run = runGame(script, new FixedRandom(0));

    check(run.text.contains("help command takes no object"),
        "help rejects unexpected arguments");
    check(run.text.contains("bread is in Echo Courtyard"),
        "hint follows a dropped quest item");
    pass();
}

private static GameRun runGame(String script, Random random) throws Exception
{
    ByteArrayOutputStream bytes = new ByteArrayOutputStream();
    PrintStream output = new PrintStream(bytes, true, "UTF-8");
    Game game = new Game(new StringReader(script), output, random);
    game.play();
    output.flush();
    return new GameRun(game, bytes.toString("UTF-8"));
}

private static void check(boolean condition, String message)
{
    if (!condition) {
        throw new AssertionError("Failed: " + message);
    }
}

private static void pass()
{
    testsRun++;
}

```

```
private static final class GameRun
{
    private final Game game;
    private final String text;

    private GameRun(Game game, String text)
    {
        this.game = game;
        this.text = text;
    }
}

private static final class FixedRandom extends Random
{
    private static final long serialVersionUID = 1L;
    private final int value;

    private FixedRandom(int value)
    {
        this.value = value;
    }

    @Override
    public int nextInt(int bound)
    {
        return Math.floorMod(value, bound);
    }
}
}
```